

Java Polymorphism (Upcasting & Downcasting)

Practical Solutions

Note

These are merely proposed solutions. You may provide other and even better ones!

Exercise 1: Demonstrating Upcasting

```
class Employee {

    protected String name;
    protected double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public void work() {
        System.out.println(name + " is working.");
    }
}

class Manager extends Employee {

    public Manager(String name, double salary) {
        super(name, salary);
    }

    @Override
    public void work() {
        System.out.println(name + " is managing a team.");
    }

    public void conductMeeting() {
        System.out.println(name + " is conducting a meeting.");
    }
}

public class UpcastingExample {

    public static void main(String[] args) {
        Employee emp = new Manager("Alice", 80000); // Upcasting
        emp.work(); // Calls overridden method in Manager
    }
}
```

```
}  
}
```

Exercise 2: Demonstrating Downcasting

```
public class DowncastingExample {  
  
    public static void main(String[] args) {  
  
        Employee emp = new Manager("Alice", 80000);  
        if (emp instanceof Manager) {  
            Manager mgr = (Manager) emp; // Downcasting  
            mgr.conductMeeting();  
        }  
    }  
}
```

Exercise 3: Upcasting & Downcasting in Collections

```
import java.util.ArrayList;  
import java.util.List;  
  
public class EmployeeCollectionExample {  
  
    public static void main(String[] args) {  
  
        List<Employee> employees = new ArrayList<>();  
        employees.add(new Employee("Bob", 50000));  
        employees.add(new Manager("Alice", 80000));  
  
        for (Employee emp : employees) {  
            emp.work();  
            if (emp instanceof Manager) {  
                ((Manager) emp).conductMeeting();  
            }  
        }  
    }  
}
```

Exercise 4: Dynamic Type Checking

```
abstract class Employee {

    protected String name;
    protected double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public abstract void work();
}

class Developer extends Employee {
    public Developer(String name, double salary) {
        super(name, salary);
    }

    @Override
    public void work() {
        System.out.println(name + " is coding.");
    }
}

public class DynamicTypeExample {
    public static void main(String[] args) {

        Employee emp = new Developer("John", 70000);
        emp.work();

        if (emp instanceof Developer) {
            System.out.println("This is a Developer.");
        }
    }
}
```

Exercise 5: Advanced Employee Management System

```
abstract class Employee {

    protected String name;
    protected double salary;
```

```

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public abstract void work();
    public abstract String getRole();
}

class Manager extends Employee {

    public Manager(String name, double salary) {
        super(name, salary);
    }

    @Override
    public void work() {
        System.out.println(name + " is managing a team.");
    }

    @Override
    public String getRole() {
        return "Manager";
    }
}

class Developer extends Employee {

    public Developer(String name, double salary) {
        super(name, salary);
    }

    @Override
    public void work() {
        System.out.println(name + " is coding.");
    }

    @Override
    public String getRole() {
        return "Developer";
    }
}

public class EmployeeSystem {

    public static void main(String[] args) {

```

```
Employee[] employees = {  
    new Manager("Alice", 90000),  
    new Developer("John", 75000)  
};  
  
for (Employee emp : employees) {  
    System.out.println(emp.name + " is a " + emp.getRole());  
    emp.work();  
}  
}
```